Sample Results Week 10

1.1) Forward pass: Calculate the values of z_1, z_2, and y

```
tensor([[1.0000],
        [0.1000],
        [0.2000]])

a = tensor([[-0.4300],
        [ 0.7600]], grad_fn=<MmBackward0>)
z = tensor([[0.0000],
        [0.7600]], grad_fn=<ReluBackward0>)
y = tensor([[1.5720]], grad_fn=<MmBackward0>)
```

1.2) Compute the mean squared error

```
tensor([[0.0916]], grad_fn=<MulBackward0>)
```

1.3) Using backpropagation, compute the gradient or the error w.r.t the weights w^(2)_2 and w^(1)_2,2

```
∂E/∂w^(2)_2 = tensor([[-0.3253]], grad_fn=<MulBackward0>)
∂E/∂w^(1)_(2,2) = tensor([[-0.1883]], grad_fn=<MulBackward0>)

∂E∂W^1 = tensor([[ 0.0000, -0.9416],
        [ 0.0000, -0.0942],
        [ 0.0000, -0.1883]])
∂E∂w^2 = tensor([[-0.4280],
        [-0.0000],
        [-0.3253]])
```

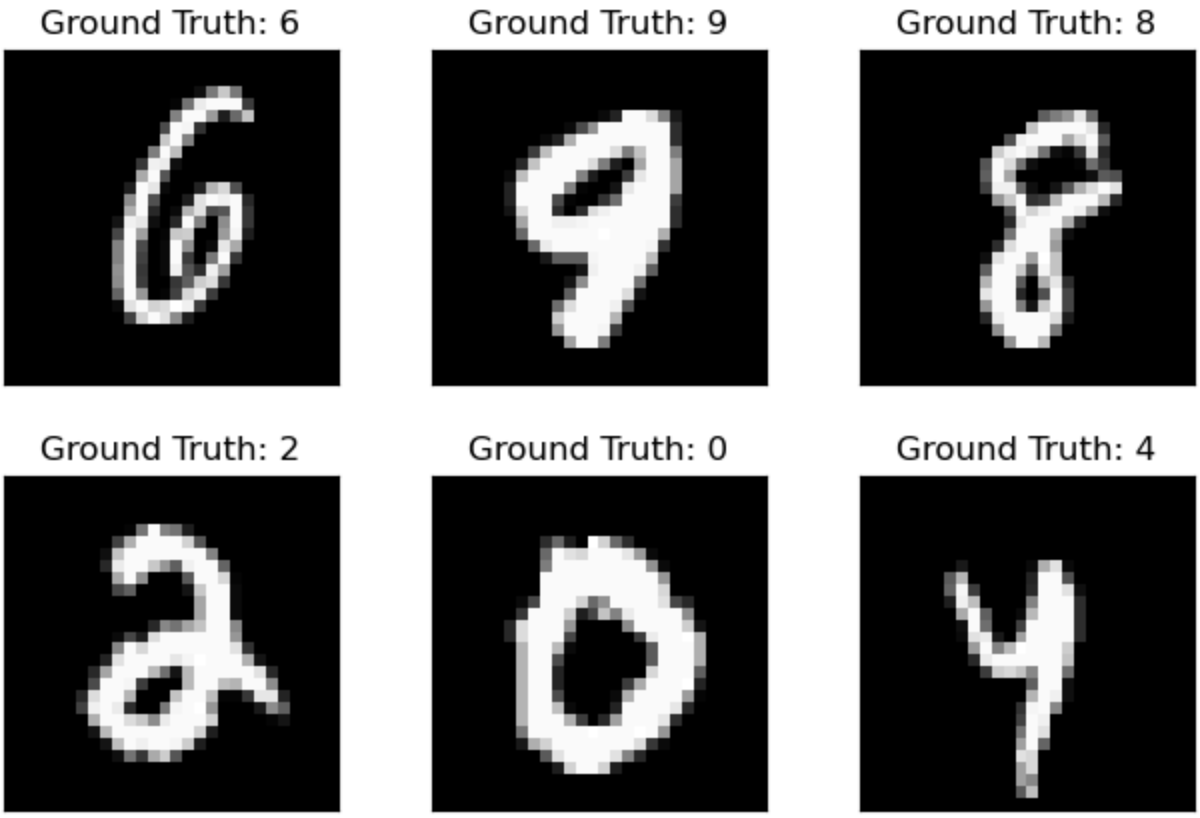
1.4) Compute the updated weights for w^(2)_2 and w^(1)_2,2

```
tensor([[2.2325]], grad_fn=<SubBackward0>)

tensor([[2.2188]], grad_fn=<SubBackward0>)
```

1. The dataset (train and test) was downloaded

```
(torch.Size([64, 1, 28, 28]), torch.Size([64]))
```



A short PyTorch primer (before you build the network)

```
torch.Size([2, 3])
tensor([[ 2.,  4.,  6.],
        [ 8., 10., 12.]])
tensor([[ 2.,  4.,  6.],
        [ 8., 10., 12.]])
torch.Size([64, 784])
```

Autograd: let PyTorch reproduce the Part 1 gradients

```
W1.grad =
tensor([[ 0.0000, -0.9416],
        [ 0.0000, -0.0942],
        [ 0.0000, -0.1883]])
w2.grad =
tensor([[-0.4280],
        [-0.0000],
        [-0.3253]])
```

2.1) Create and train a 2-layer MLP

```
Train Epoch: 1 [0/60000 (0%)]   Loss: 2.320454
Train Epoch: 5 [57600/60000 (96%)]      Loss: 0.425199
Test set: Average loss: 0.9177, Accuracy: 8255/10000 (83%)
Test set: Average loss: 0.5869, Accuracy: 8662/10000 (87%)
Test set: Average loss: 0.4728, Accuracy: 8851/10000 (89%)
Test set: Average loss: 0.4154, Accuracy: 8922/10000 (89%)
Test set: Average loss: 0.3807, Accuracy: 8982/10000 (90%)
```